Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 14 : frais de port et commande

1. Gestion des villes et des frais de port (tarifs de livraison). Dans notre application, les frais de port dépendent des villes de livraison.

   TRAVAIL A FAIRE POUR LES ETUDIANTS :

   - Créer une entité « City » dotée de 3 attributs : « name » (chaîne de 255 caractères), « zipCode » (code postal - chaîne de 5 caractères) et « shippingCost » (frais de port – type Decimal avec 2 chiffres après la virgule).
   - Préparer et exécuter la migration pour créer la table « city » dans la BDD.
   - Créez le CRUD pour l'entité « City ».
   - Mettre en forme toutes les vues qui concernent les produits (avec Bootstrap).
   - Ajouter des messages flash lors de l'ajout la modification et la suppression d'une ville.
   - Réserver la gestion des villes aux utilisateurs ayant le rôle « ROLE_EDITOR ».
   - Ajouter la gestion des villes dans la navbar (uniquement pour les administrateurs et les éditeurs). Rappel : les administrateurs sont aussi des éditeurs.

2. **Ajout de la commande.**

   Nous allons utiliser une entité « Order » qui va contenir toutes les informations liées à la commande. Chaque « Order » sera relié à une ville de livraison (city).

   Commencez par créer l'entité « Order » : `symfony console make:entity Order`

```
PS C:\Symfony\e-commerce> symfony console make:entity Order
  created: src/Entity/Order.php
  created: src/Repository/OrderRepository.php

  Entity generated! Now let's add some fields!
  You can always add more fields later manually or by re-running this command.

  New property name (press <return> to stop adding fields):
  > firstName

  Field type (enter ? to see all types) [string]:
  >

  Field length [255]:
  >

  Can this field be null in the database (nullable) (yes/no) [no]:
  >

  updated: src/Entity/Order.php

  Add another property? Enter the property name (or press <return> to stop adding fields):
  > lastName

  Field type (enter ? to see all types) [string]:
  >

  Field length [255]:
  >

  Can this field be null in the database (nullable) (yes/no) [no]:
  >

  updated: src/Entity/Order.php

  Add another property? Enter the property name (or press <return> to stop adding fields):
  > phoneNumber

  Field type (enter ? to see all types) [string]:
  >

  Field length [255]:
  >

  Can this field be null in the database (nullable) (yes/no) [no]:
  >

  updated: src/Entity/Order.php

  Add another property? Enter the property name (or press <return> to stop adding fields):
  > adress

  Field type (enter ? to see all types) [string]:
  >

  Field length [255]:
  >

  Can this field be null in the database (nullable) (yes/no) [no]:
  >

  updated: src/Entity/Order.php
```

```
  Add another property? Enter the property name (or press <return> to stop adding fields):
  > city

  Field type (enter ? to see all types) [string]:
  > ManyToOne

  What class should this entity be related to?:
  > City

  Is the Order.city property allowed to be null (nullable)? (yes/no) [yes]:
  >

  Do you want to add a new property to City so that you can access/update Order objects from it - e.g. $city->getOrders()
? (yes/no) [yes]:
  >

  A new property will also be added to the City class so that you can access the related Order objects from it.

  New field name inside City [orders]:
  >

  updated: src/Entity/Order.php
  updated: src/Entity/City.php
```

```
Add another property? Enter the property name (or press <return> to stop adding fields):
> createdAt

Field type (enter ? to see all types) [datetime_immutable]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Order.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>


 Success!

Next: When you're ready, create a migration with symfony.exe console make:migration
```

3. **Préparez et exécutez la migration pour mettre à jour la BDD :**

   ```
   symfony console make:migration
   symfony console doctrine:migrations:migrate
   ```

4. **Créez le contrôleur pour les commandes :**

   ```
   symfony console make:controller OrderController
   ```

5. **Créez un nouveau formulaire « OrderFormType » en utilisant la commande :**

   ```
   symfony console make:form
   ```

   Liez le nouveau formulaire à l'entité « Order» :

```
PS C:\Symfony\e-commerce> symfony console make:form OrderType

 The name of Entity or fully qualified model class name that the new form will be bound to (empty for none):
 > Order

 created: src/Form/OrderTypeForm.php

 Success!

 Next: Add fields to your form and start using it.
 Find the documentation at https://symfony.com/doc/current/forms.html
```

6. **Modification du fichier « src\Form\OrderTypeForm.php ».**

   Commencez par importer la bibliothèque « EntityRepository » : `use Doctrine\ORM\EntityRepository;`

   Modifiez ensuite le code pour afficher les champs à remplir. Attention, le champ `'createdAt'` ne doit pas apparaître car il sera valorisé automatiquement lors de l'enregistrement de la commande (mettez le code concerné en commentaire) :

```php
public function buildForm(FormBuilderInterface $builder, array $options): void
{
        $builder
            ->add('firstName')
            ->add('lastName')
            ->add('phoneNumber')
            ->add('adress')
            //->add('createdAt', null, [
            //    'widget' => 'single_text',
            //])
            ->add('createdAt', null, [
```

```
            'widget' => 'single_text',
        ])
        ->add('city', EntityType::class, [
            'class' => City::class,
            //'choice_label' => 'name',
            'choice_label' => function($city) {
                return $city->getZipCode() . " - " . $city->getName();
            },
            'query_builder' => function (EntityRepository $er) {
                return $er->createQueryBuilder('c')
                  ->orderBy('c.zipCode', 'ASC')
                  ->addOrderBy('c.name', 'ASC');
            },
        ])
    ;
}
```

Observez le code qui permet d'afficher la liste des villes : on regroupe ici deux champs (zipCode et name) dans la liste déroulante et on trie la liste dans l'ordre alphabétique (d'abord par code postal puis par nom lorsque le code postal est identique).

7. **Créez un fichier « templates\order_form.html.twig » avec le code suivant :**

```
{{ form_start(form) }}
    <label for="">Prénom : </label>
{{ form_widget(form.firstName,{'attr':{'class':'form form-control',
'placeholder':"prénom"}}) }}
    {{ form_errors(form.firstName) }}
    <br>
    <label for="">Nom: </label>
    {{ form_widget(form.lastName,{'attr':{'class':'form form-control'}}) }}
    <br>
    <label for="">Téléphone : </label>
    {{ form_widget(form.phoneNumber,{'attr':{'class':'form form-control'}}) }}
    <br>
    <label for="">Adresse : </label>
    {{ form_widget(form.adress,{'attr':{'class':'form form-control'}}) }}
    <br>
    <label for="">Ville : </label>
    {{ form_widget(form.city,{'attr':{'class':'form form-control'}}) }}
    <button class="btn btn-primary">{{ button_label|default('Payer') }}</button>
{{ form_end(form) }}
```

8. Modifications du contrôleur « src\Controller\OrderController.php ».

Commencez par ajouter les bibliothèques suivantes :

```
use App\Entity\Order;
use App\Form\OrderTypeForm;
use App\Repository\CategoryRepository;
use App\Repository\ProductRepository;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Session\SessionInterface;
use Doctrine\ORM\EntityManagerInterface;
```

Modifiez ensuite le code de la route « /Order » :

```
#[Route('/order', name: 'app_order')]
public function index(Request $request, CategoryRepository $categoryRepository,
ProductRepository $productRepository, SessionInterface $session,
EntityManagerInterface $entityManager): Response
{
        $cart = $session->get('cart', []);
        $cartWithData = [];

        // On boucle dans le tableau $cart et on récupère les données de chaque produit
        foreach ($cart as $id => $quantity) {
            $product = $productRepository->find($id);

            if ($product) {
                // On ajoute le produit et la quantité au tableau $cartWithData
                $cartWithData[] = [
                    'product' => $product,
                    'quantity' => $quantity
                ];
            }
        }

        // Calcul du prix total du panier
        $total = array_sum(array_map(function ($item) {
            return $item['product']->getPrice() * $item['quantity'];
        }, $cartWithData));

        $order = new Order();
        $form = $this->createForm(OrderTypeForm::class, $order);
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid())
        {
            $order->setCreatedAt(new \DateTimeImmutable());
            $entityManager->persist($order);
```

```
            $entityManager->flush();

            return $this->redirectToRoute('app_home', [], Response::HTTP_SEE_OTHER);
        }

        return $this->render('order/index.html.twig', [
            'controller_name' => 'OrderController',
            'form' => $form,
            'total' =>$total,
            'categories' => $categoryRepository->findAll(),
        ]);
    }
```

Remarque : comme indiqué lors de la création du formulaire, la propriété « createdAt » est directement valorisée dans le contrôleur à la date et l'heure de la soumission du formulaire avec l'instruction « `$order->setCreatedAt(new \DateTimeImmutable());` ».

9. **Mise à jour de la vue « templates\order\index.html.twig ».**

Modifiez la vue pour afficher le formulaire ainsi que le prix total :

```
{% extends 'base.html.twig' %}

{% block title %}Commande{% endblock %}

{% block body %}
   <div class = "container">
        <br>
        <h1>Commande</h1>
        <br>
        <div class="col-4">
            <h4>Montant total : {{ total|number_format(2, '.', ',') }} €</h4>
        </div>
        {{ include('order/_form.html.twig') }}
        <br>
        <a class="btn btn-dark" href="{{ path('app_cart') }}">Retour</a>

    </div>
{% endblock %}
```

10. Ajout du lien pour passer la commande à partir du panier.

Ouvrez la vue « templates\cart\index.html.twig » et modifiez le lien du bouton « Valider et Payer » :

```
<div class="d-flex justify-content-end mt-3">
    <a data-turbo="false" href= »{{ path('app_order') }} » class= »btn btn-primary »>
    Valider et Payer</a>
     
    <a href= »{{ path('app_cart_clear')}} » class= »btn btn-danger »>Vider le panier</a>
</div>
```

Remarque : le paramètre `data-turbo="false"` permet de forcer le rechargement de la page.